CS & IT ENGINEERING



Algorithms

Heap Algorithms



Lecture No.- 01

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Graph Traversals

Topics to be Covered



- Topic
- Topic
- Topic

Heap Algo

- Intro
- Operations
  - insert
    - Creation
  - delete



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

PYQs on Graph Traversals



Topic : Heap Algorithms

#Q. In a DF-traversal of a graph 'a' with n-vertices, 'k' edges are marked as tree edges, the number of connected components of 'G' is:

$k = n-1 \rightarrow (1)$ Comp

A. k ✗

B. k + 1 ✗

C. (n – k – 1) ✗

D. n – k $\quad n-(n-1)=1$

D



Shortcut:

n vertices

1 Component

DFS Tree → (n-1) K = n-1



Topic : Heap Algorithms

tricky

#Q. DFS is performed on a directed acyclic graph. D(u) is discovery time and f(u) is finishing time. Which is true for all edges (u, v) in the graph?

25 J.

1/4 2/3

u → v

3/4 1/2

u → v

A d[u] < d[v]

B d[u] < f[v]

C f[u] < f[v] ✗

D f[u] > f[v] ✓

u → v



Topic : Heap Algorithms

MCQ

#Q. Consider an undirected graph (unweighted). If BFS of G is done from a node 'r' let d(r, u) and d(r, v) be the lengths of the shortest paths from r to u and v. If 'u' is visited before 'v', during the traversal, then which is true?

46/.

A $d(r, u) < d(r, v)$ →

B $d(r, u) > d(r, v)$ ✗

C $d(r, u) \leq d(r, v)$ → most correct

D None

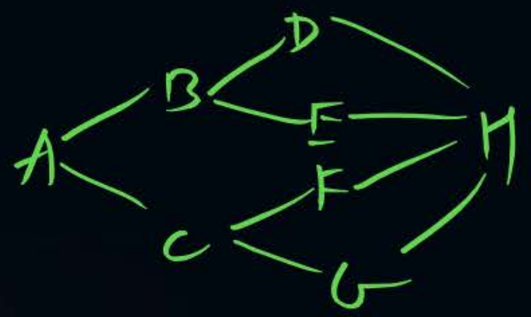
Topic : Heap Algorithms

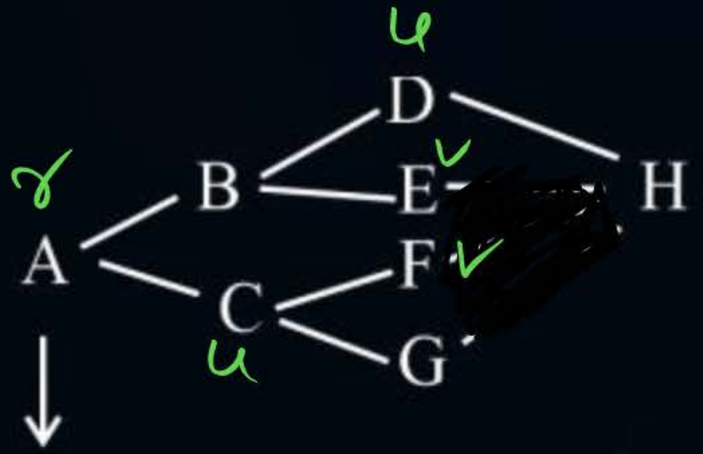
Example:

$A \rightarrow D : 2$

$A \rightarrow E : 2$

$d(r, u) = d(r, v)$





FIFO Queue:

Start

Node	B	C	D	E	F	G	H
Parent	A	A	B	B	C	C	D

$d(r,u) < d(r,v)$

1 2

AB CDEFGH

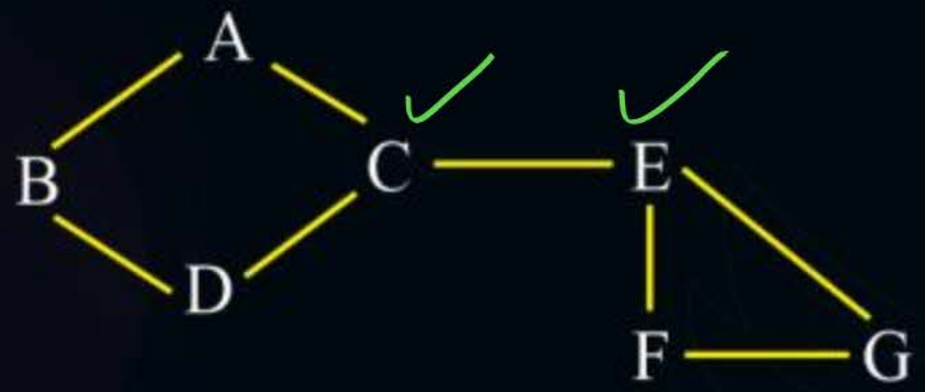


Topic : Heap Algorithms

#Q. Given a graph G, it has A articulation points and B bi-connected components, then what is the value of $(A)^B$?

A = 2
B = 3

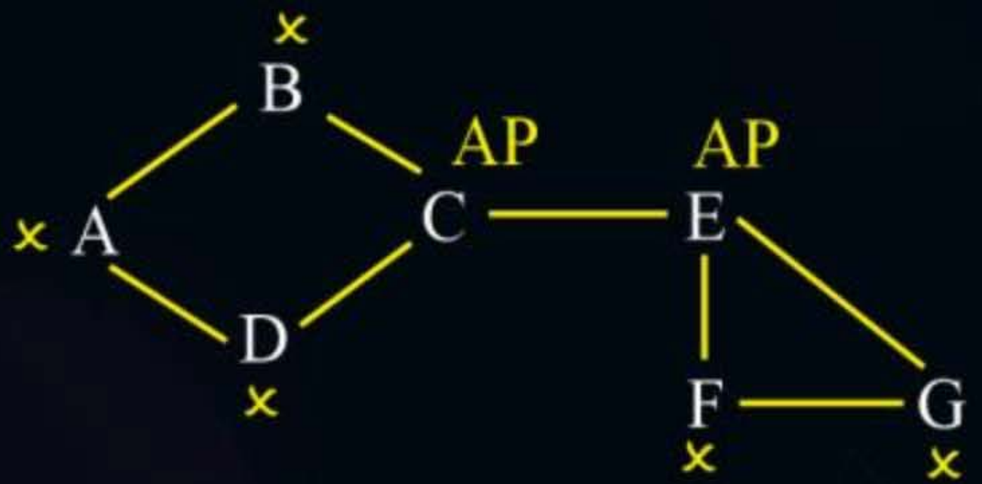
$2^3 = 8$



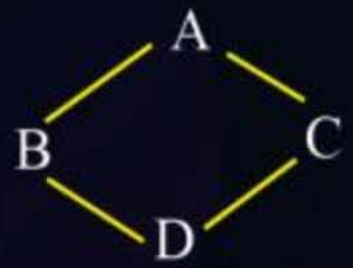


Topic : Heap Algorithms

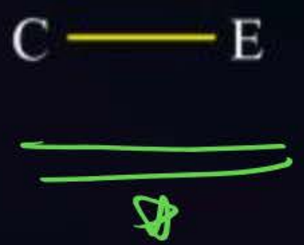
Solution:



BCC1

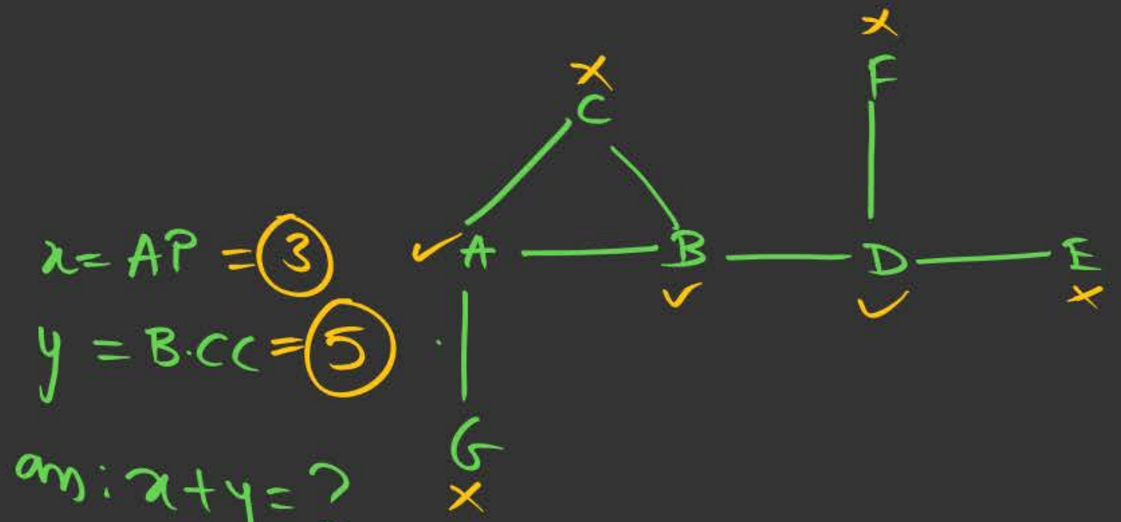


BCC2



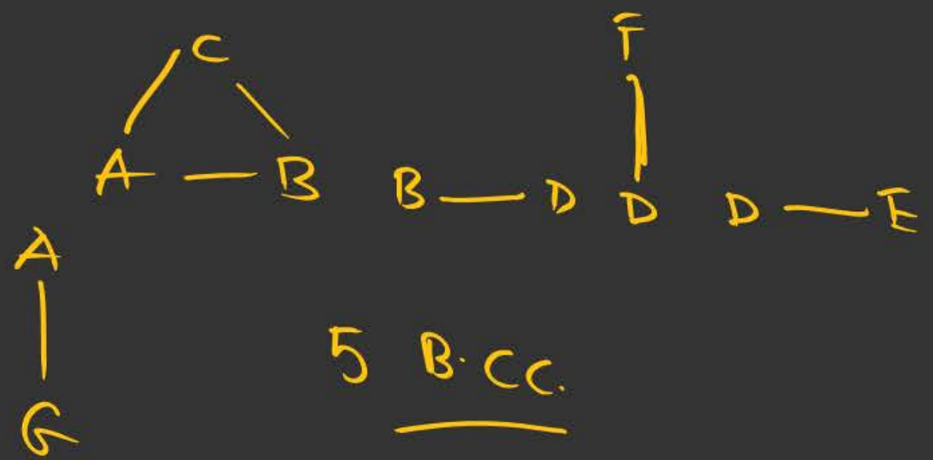
BCC3





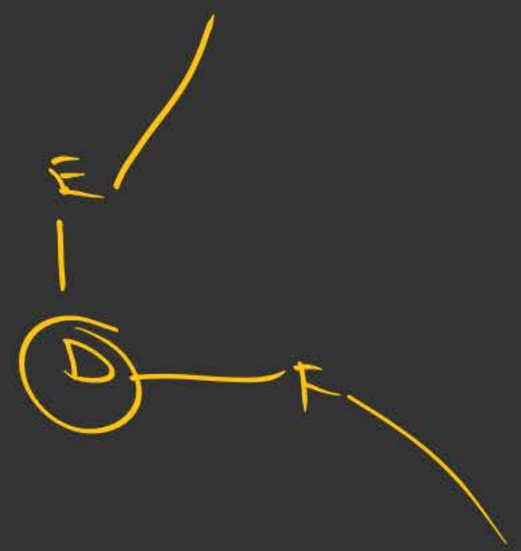
3+5

= 8 ✓



5 B·CC.

42.





Topic : Heap Algorithms

#Q. Which of the following are valid BFS sequences?



(E) None

A M N O P Q R

B N Q M P O R

C Q M N P R O

D Q M N P O R

m n q r

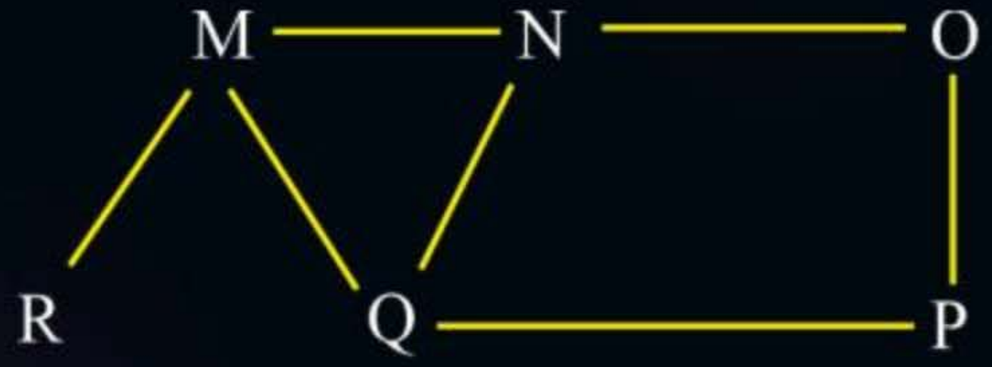
n m q o

Ans : C



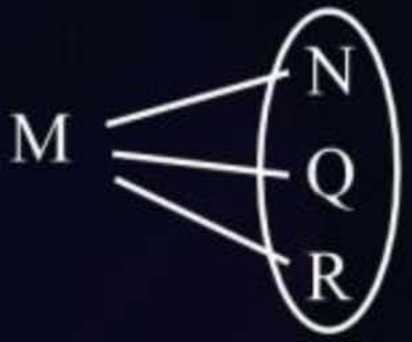
Topic : Heap Algorithms

Solution:



(1) M N O P Q R

(2) N Q M P O R

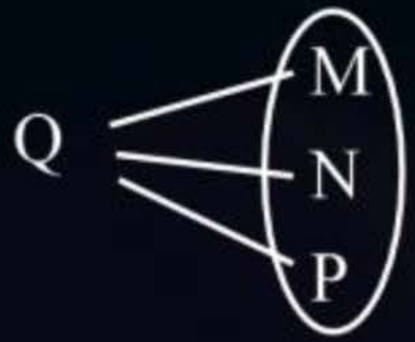






Topic : Heap Algorithms

(3) Q M N P R O



(4) Q M N P O R

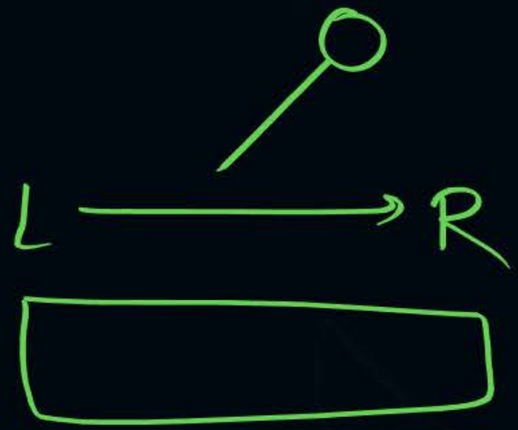




Topic : Heap Algorithms

Heaps:

- Complete Binary Tree



Types of Heaps: V. Imp

(1) Max Heap (default)

(2) Min Heap



Topic : Heap Algorithms

(1) Max Heap (default):

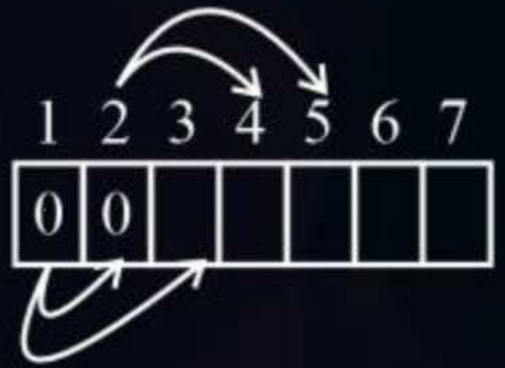
- A max-Heap is a Complete Binary Tree with a property that the value at each node is greater than or equal to all of its children.

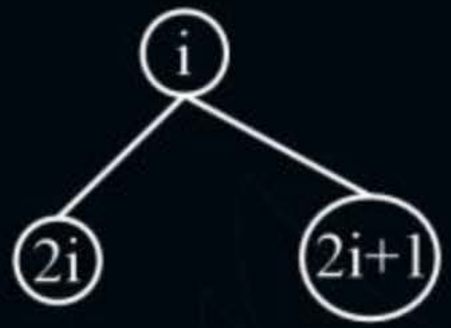


Topic : Heap Algorithms

Array representation of a Complete Binary Tree:

$i \geq 1$ (1 based indexing)





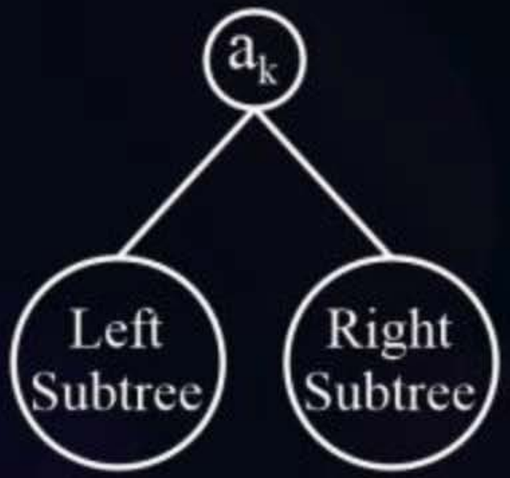


Topic : Heap Algorithms

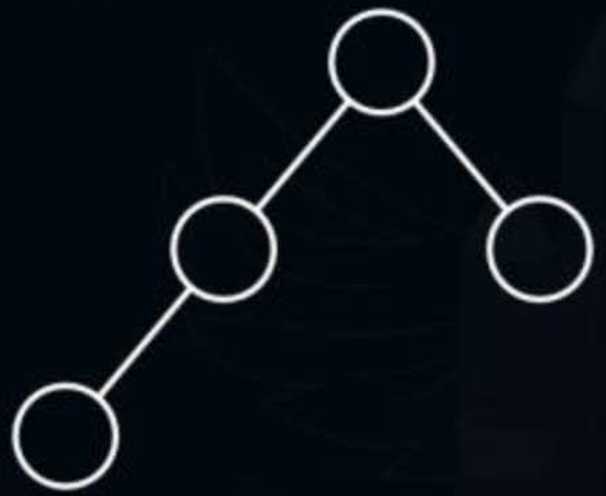
(1) Max Heap (default):

$a_k \geq$ all values in the left subtree and the right subtrees.

General:



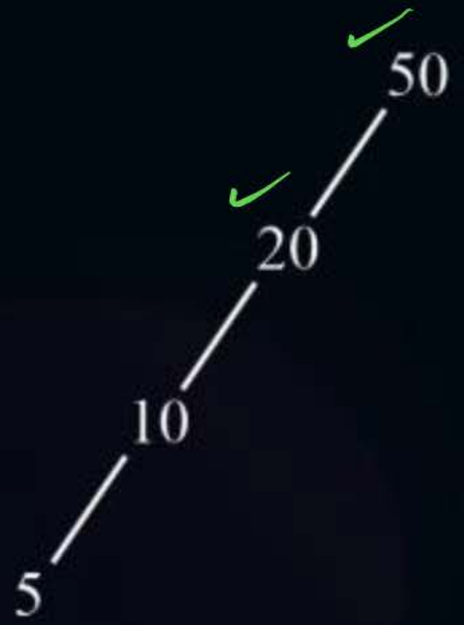
Complete BT (CBT)





Topic : Heap Algorithms

Example 1:

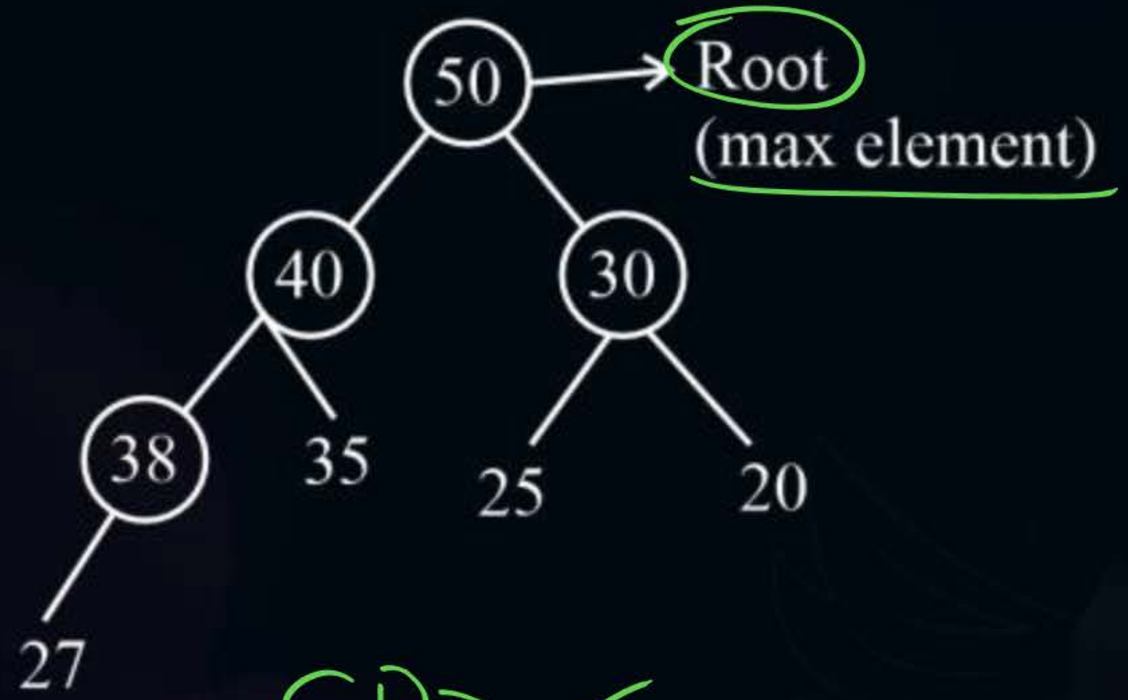


Not a Complete Binary Tree



Topic : Heap Algorithms

Example 2:



Valid Max Heap

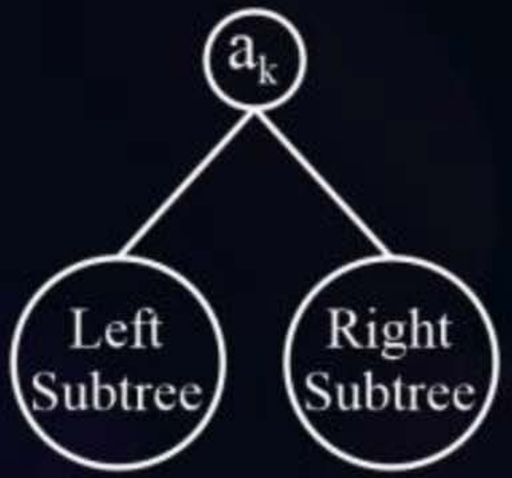


Topic : Heap Algorithms

2) Min Heap:

$a_k \leq$ all values in the left subtree and the right subtrees.

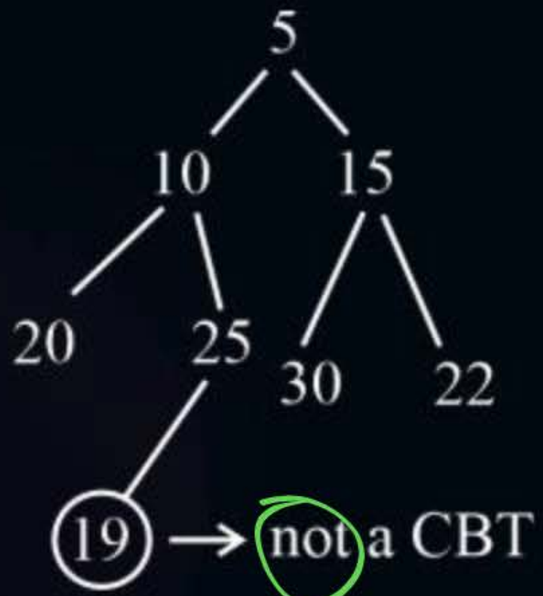
General:





Topic : Heap Algorithms

Example 1:

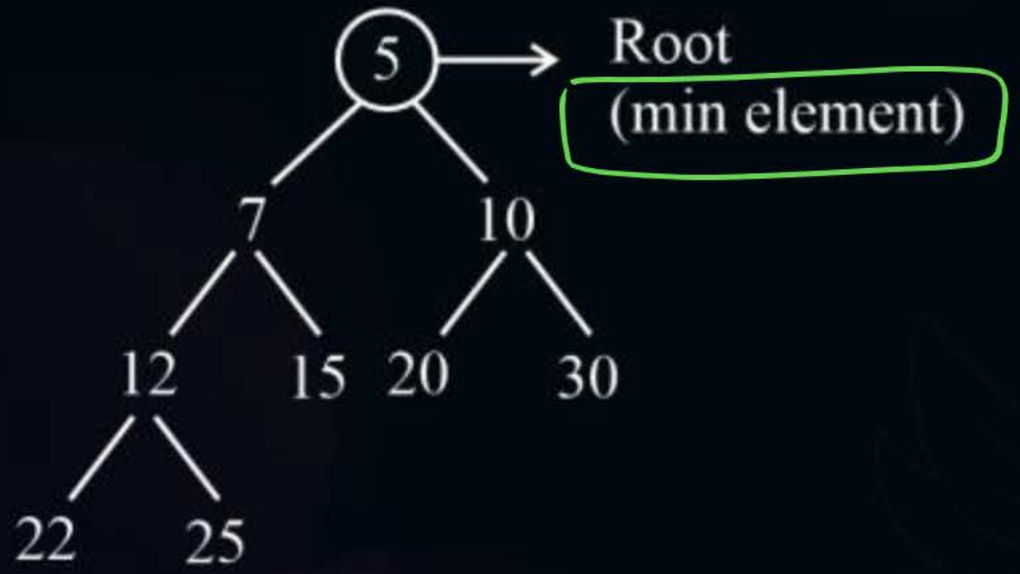


Not a Complete Binary Tree



Topic : Heap Algorithms

Example 2:



Valid Min Heap



Topic : Heap Algorithms

Construction of a Heap:

(1) Insertion method

(2) Heapify/Build Heap method



Topic : Heap Algorithms

Insertion method:

Steps:

(1) Start with an empty tree.

(2) Insert one element at a time, always to a valid Heap of elements that are inserted so far.

(A new element is ~~insulted~~ inserted always to a valid Heap)

Bottom-Up ↑



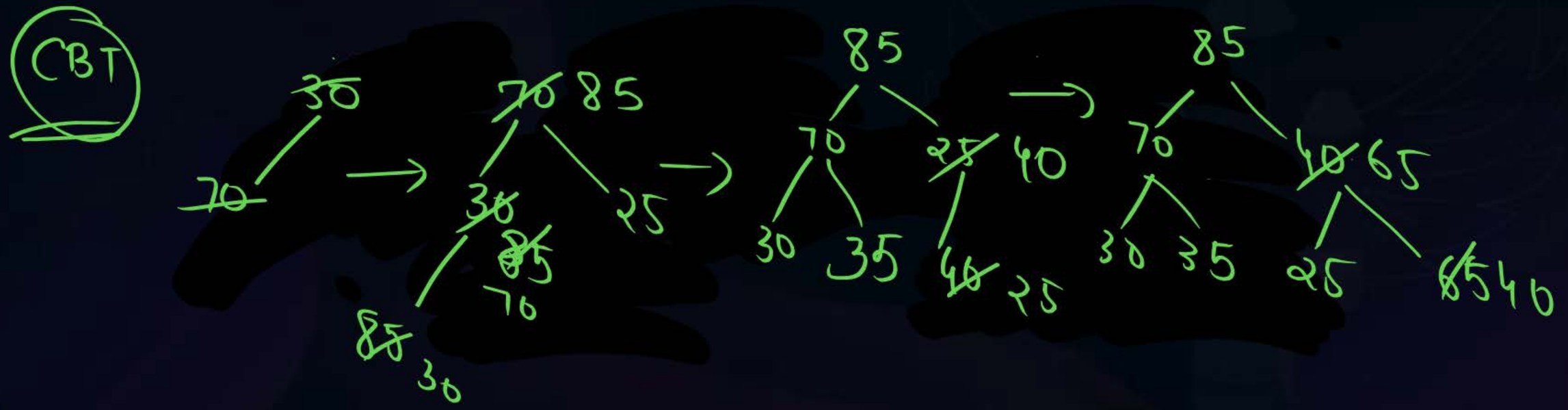
Topic : Heap Algorithms

Example:

A = [30, 70, 25, 85, 35, 40, 65]

Construct a Max-Heap:

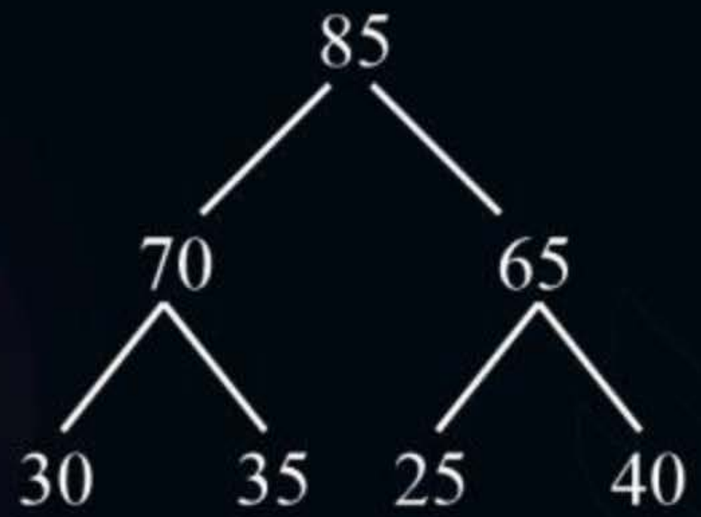
- Insert in CBT ~~fusion.~~ Fashion





Topic : Heap Algorithms

Final Max Heap:



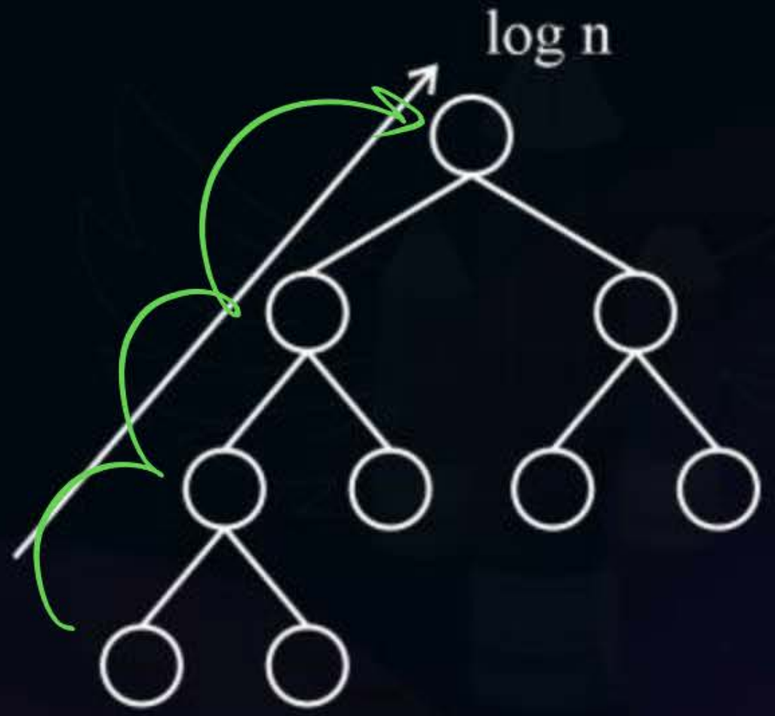


Topic : Heap Algorithms

Important Note:

Time Complexity of insert Operation:

- Given a Heap of 'n' elements, the time complexity of inserting an element into this Heap is O(log n).
- Height of a CBT of n elements
  → always O(log n)



n elems : Heap Creation





Topic : Heap Algorithms

Time Complexity of Heap creation using Insertion method:

(1) Best Case: B.C

- I/p array already sorted in decreasing order.
- Every element will need only 1 comparison (with its parent)

→ $n * 1 = O(n) = \Omega(n)$





Topic : Heap Algorithms

Time Complexity of Heap creation using Insertion method:

(2) Worst Case:

- I/p is already sorted in increasing order.
- Every element will move to the top, level-by-level.

→ $O(\log_2 n)$

logic

Hence, for n elements,

Overall TC = $O(n * \log_2 n)$





Topic : Heap Algorithms

Bonus:

Logical/mathematical approach to determine worst case complexity of Heap Creation using Insertion method.



Topic : Heap Algorithms

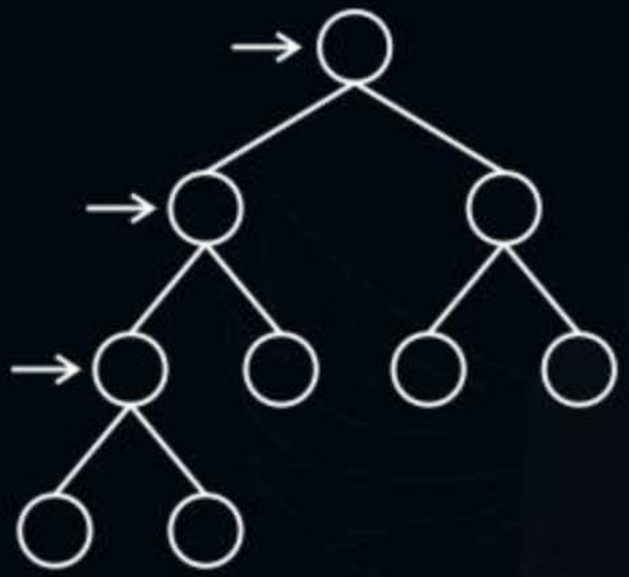
(1) Max no. of nodes at any level 'i' of a Binary Tree (Root at level 1) $=2^{(i-1)}$

$1 \rightarrow 1 \rightarrow 2^0$

$2 \rightarrow 2 \rightarrow 2^1$

$3 \rightarrow 4 \rightarrow 2^2$

$4 \rightarrow 8 \rightarrow 2^3$





Topic : Heap Algorithms

(2) The no. of level comparisons for a node that is getting inserted at level i

$= (i - 1)$

(3) The total no. of level comparisons for all nodes at level 'i'.

$= 2^{(i-1)} * (i - 1)$





Topic : Heap Algorithms

(4) Overall TC to create a Heap of 'n' elements

= No. of comparisons for all the nodes at all the level (assume k levels)

$$\sum_{i=1}^{k} 2^{(i-1)} * (i-1) = \sum_{i=1}^{k} i * 2^{(i-1)} - \sum_{i=1}^{k} 2^{(i-1)}$$

$$= \frac{1}{2}\left[\sum_{i=1}^{k} i * 2^{i} - \sum_{i=1}^{k} 2^{i}\right]$$



Topic : Heap Algorithms

Important:

$$\sum_{i=1}^{k} i * 2^i = (k-1) * 2^{(k+1)} + 2 \qquad \text{... (i)}$$

$$\sum_{i=1}^{k} 2^i = \frac{a(r^n - 1)}{r - 1} = \frac{2(2^k - 1)}{2 - 1} = 2(2^k - 1) \qquad \text{... (ii)}$$



Topic : Heap Algorithms

Total Comparisons

$$= \frac{1}{2}\left[\sum_{i=1}^{k} i * 2^i - \sum_{i=1}^{k} 2^i\right]$$

using Results from (i) and (ii)

$$= \frac{1}{2}\left[\left((k-1) * 2^{(k+1)} + 2\right) - \left(2\left(2^k - 1\right)\right)\right]$$

$$= \frac{1}{2}\left[2 \times 2^k \times k - 2 \times 2^k + 2 - 2 \times 2^k + 2\right]$$



Topic : Heap Algorithms

$$= \frac{1}{2}[2 \times 2^k \times k - 2 \times 2 \times 2^k + 2 \times 2]$$

$$= [2^k \times k - 2 \times 2^k + 2] \qquad \rightarrow O(k * 2^k)$$

$$n = 2^k \Rightarrow k = \log_2 n \Rightarrow \boxed{O(n * \log_2 n)}$$



Topic : Heap Algorithms

```
Procedure INSERT(A, n)
Integer i, j, n;
j = n; i ← ⌊n/2⌋; item ← A(n)
while i > 0 and A(i) < item do
    A(j) ← A(i)// move the parent down//
    j ← i; i ← ⌊i/2⌋
repeat
A(j) ← item    //a place for A(n) is found//
end INSERT
for i ← 2 to n do
    call INSERT (A, i)
repeat
```



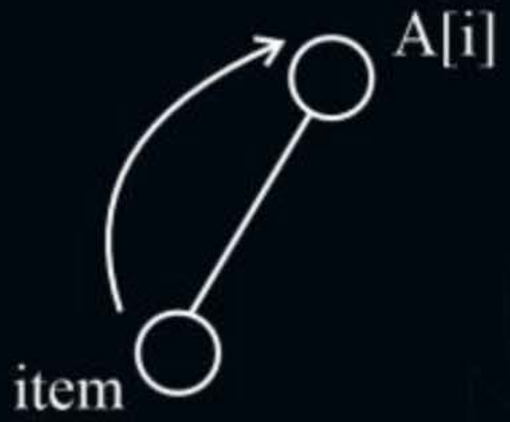


Topic : Heap Algorithms

Heap Creation using Insertion method:

- A[i] < item
- parent < child

swap





Topic : Heap Algorithms

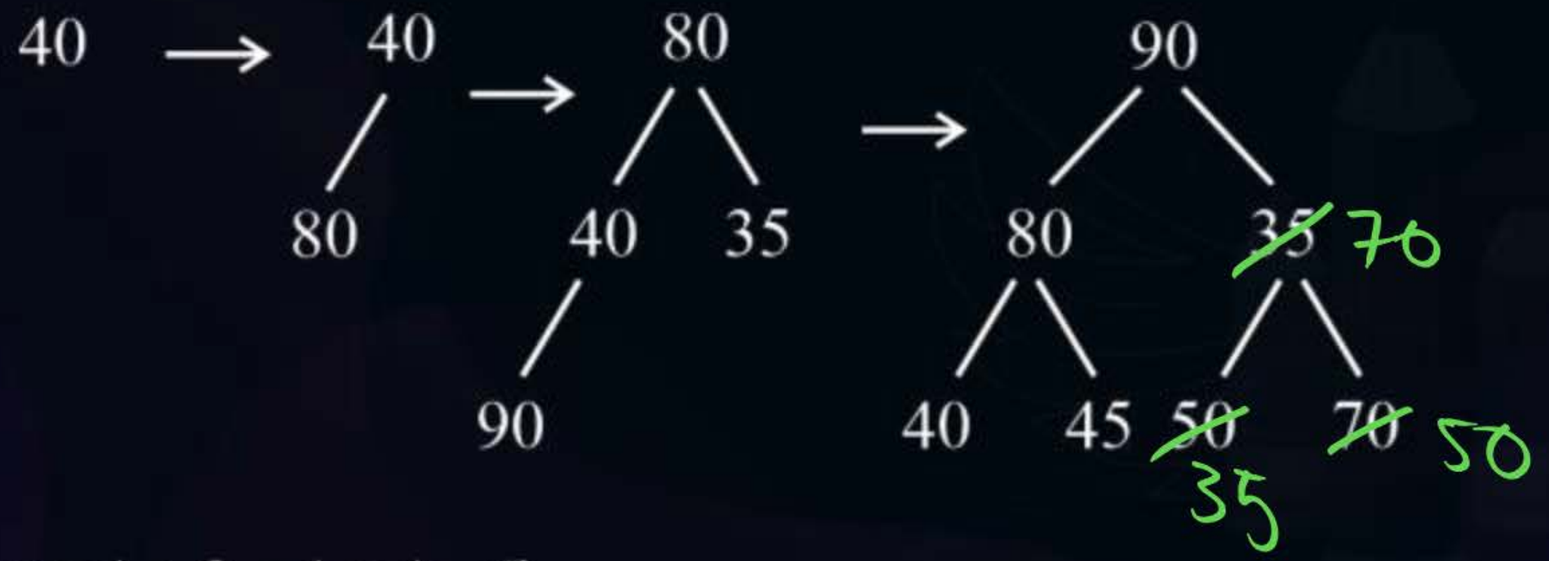
95%

71%

Test: Given A : [40, 80, 35, 90, 45, 50, 70]

#Q. Create a max-Heap using insertion method.

How many swaps happened during the process?



Total swaps = 1 + 2 + 1 + 1 = 5

Heapify



Topic : Heap Algorithms

Heap Creation using Heapify/Build Heap Method:

Steps:

1) Complete Binary Tree already exists of n elements.
2) Make level-by-level adjustments to each node to get to the final Heap.
3) Adjust (Top-Down) a node to move it to tis correct position.

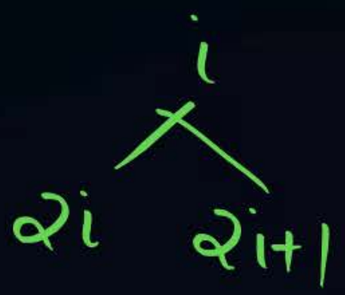


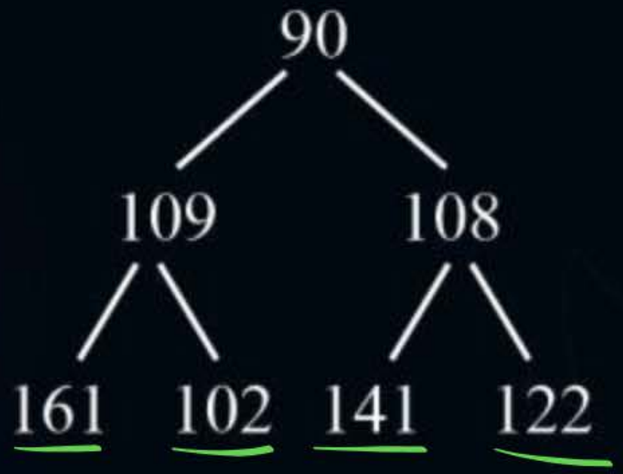


Topic : Heap Algorithms

Example: A : 90, 109, 108, 161, 102, 141, 122 A[n]

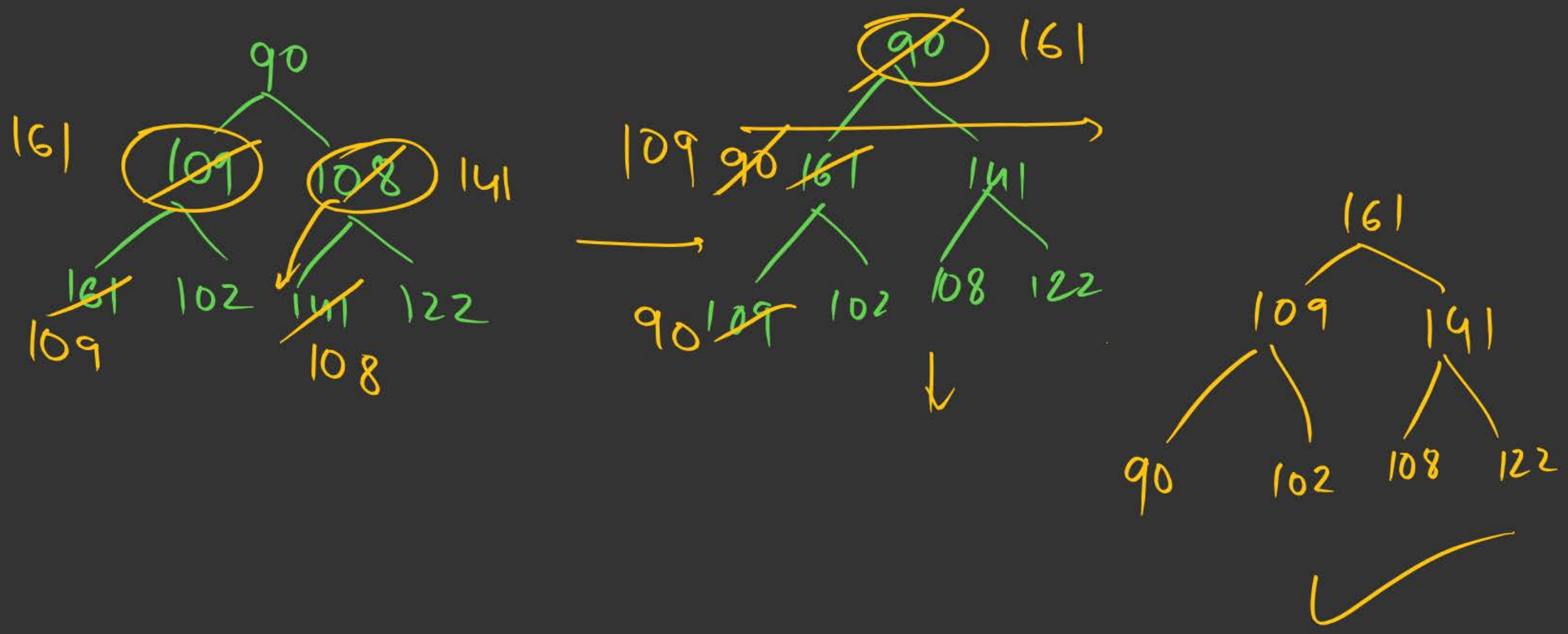
Given CBT is:





Important:

- Check for adjustments from the 2nd last level (n/2 nodes ~~at last level~~ are leaf)


90
161
109
108
141
161
102
141
122
109
108
109
90
161
141
90
109
102
108
122
161
109
141
90
102
108
122



Topic : Heap Algorithms

Important:

For i/p : A[n]

Heapify → TC : O(n) for Heap Creation

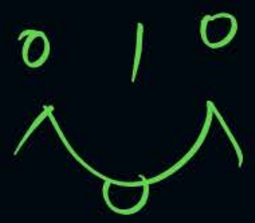


2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

PYQs on Traversals

Heaps



THANK - YOU